1. 定时器-QTimer的使用

1.1 实时显示系统时间

借助QLabel显示系统时间

实时： 引入QTimer定时器 要使用定时器： 超时时间设置、定时器启动、定时器信号

系统时间获取： QDateTime类的使用

```
需求：创建一个界面，实时显示当前时间

要求：每秒更新一次显示时间

需要的组件： 标签 QLabel

实现方式：手写代码完成开发
```

1.2 倒计时实现

借助QLCDNumber类显示，它提供了一个液晶显示屏（LCD）数字显示部件，可以用display()方法来显示数字。

每一秒都要更新： 引入QTimer定时器 超时时间设置、定时器启动、定时器信号

倒计时： 需要有计数值， 每一秒数值都要-1， 然后要转成时间的格式显示， 引入 QTime类 将数值类型转成时间格式

```
需求：创建一个界面，用于倒计时

要求：每秒更新一次

需要的组件： 标签 QLabel，按钮QPushButton,对话框QMessageBox

实现方式：手写代码完成开发
```

传清远见|元宇宙实验中心
yyzlab.com.cn

2. 总结

2.1 QTimer

在 PyQt5 中，`QTimer` 是一个用于定时操作的类。它属于 `PyQt5.QtCore` 模块，主要用于在指定的时间间隔内执行某些任务或触发某些事件。`QTimer` 是实现定时器功能的核心工具，常用于 GUI 应用程序中的周期性任务、延迟操作等。

`QTimer` 的主要作用是定时触发某个操作或事件。它可以用于以下场景：

- **周期性任务**：如每隔一段时间更新界面、检查状态等。
- **延迟操作**：如延迟一段时间后再执行某个操作。
- **超时处理**：如在一定时间内未收到响应时触发超时事件。

2.1.1 `QTimer` 的导入方式

```
from PyQt5.QtCore import QTimer
```

2.1.2 常用方法

2.1.2.1 创建 `QTimer` 对象

- `QTimer(parent=None)`：创建一个定时器对象。

```
timer = QTimer()
```

2.1.2.2 设置定时器的时间间隔

- `setInterval(int msec)`：设置定时器的时间间隔（以毫秒为单位）。

```
timer.setInterval(1000)  # 设置定时器每 1 秒触发一次
```

2.1.2.3 启动和停止定时器

- `start()`：启动定时器。

```
timer.start()  # 启动定时器
```

- `stop()`：停止定时器。

```
timer.stop()  # 停止定时器
```

2.1.2.4 单次触发定时器

- `singleShot(int msec, callable)`：在指定的时间后触发一次操作。

```
QTimer.singleShot(2000, lambda: print("2 秒后执行"))  # 延迟 2 秒后执行打印操作
```

2.1.2.5 连接定时器信号

- `timeout.connect(slot)`：当定时器超时时触发的信号。

```
def on_timeout():
    print("定时器超时")

timer.timeout.connect(on_timeout)
```

2.1.2.6 检查定时器是否正在运行

- `isActive()`：检查定时器是否正在运行。

```
if timer.isActive():
    print("定时器正在运行")
```

华清远见|元宇宙实验中心
yyzlab.com.cn

2.1.2.7 获取剩余时间

- `remainingTime()`：获取定时器距离下次超时的剩余时间（以毫秒为单位）。

```
remaining_time = timer.remainingTime()
print(f"剩余时间: {remaining_time} 毫秒")
```

2.1.3 典型应用场景

- **周期性任务**：可以每隔一段时间执行某个操作，如更新界面、检查状态等。
- **延迟操作**：可以延迟一段时间后再执行某个操作，如延迟显示提示信息。
- **超时处理**：可以在一定时间内未收到响应时触发超时事件，如网络请求超时。

2.2 QDateTime

在 PyQt5 中，`QDateTime` 是一个用于处理日期和时间的类。它属于 `PyQt5.QtCore` 模块，主要用于表示和操作日期与时间的组合。`QDateTime` 可以表示某个特定的时间点，并且支持多种操作，如日期时间的加减、比较、格式化等。

`QDateTime` 的主要作用是处理日期和时间的组合。它可以用于以下场景：

- **获取当前日期和时间**：可以获取系统当前的日期和时间。
- **日期时间的加减**：可以对日期时间进行加减操作，例如增加或减少天数、小时数等。
- **日期时间的比较**：可以比较两个日期时间的先后顺序。
- **日期时间的格式化**：可以将日期时间转换为字符串，或者从字符串解析日期时间。
- **时区处理**：可以处理不同时区的日期时间。

2.2.1 `QDateTime` 的导入方式

```
from PyQt5.QtCore import QDateTime
```

2.2.2 常用方法

2.2.2.1 创建 `QDateTime` 对象

- `QDateTime()`：创建一个空的 `QDateTime` 对象。

```
datetime = QDateTime()
```

- `QDateTime(QDate date, QTime time)`：通过指定的日期和时间创建 `QDateTime` 对象。

```
from PyQt5.QtCore import QDate, QTime

date = QDate(2023, 10, 1)
time = QTime(12, 0, 0)
datetime = QDateTime(date, time)  # 创建 2023 年 10 月 1 日 12:00:00 的日期时间对象
```

- `currentDateTime()`：获取系统当前的日期和时间。

```
current_datetime = QDateTime.currentDateTime()
print(f"当前日期时间: {current_datetime.toString()}")
```

华清远见先学自实验中心 yyzlab.com.cn

2.2.2.2 日期时间的加减

- `addDays(int days)`：在当前日期时间的基础上增加或减少天数。

```
new_datetime = current_datetime.addDays(7)  # 增加 7 天
```

- `addMonths(int months)`：在当前日期时间的基础上增加或减少月数。

```
new_datetime = current_datetime.addMonths(1)  # 增加 1 个月
```

- `addYears(int years)`：在当前日期时间的基础上增加或减少年数。

```
new_datetime = current_datetime.addYears(1)  # 增加 1 年
```

- `addSecs(int secs)`：在当前日期时间的基础上增加或减少秒数。

```
new_datetime = current_datetime.addSecs(3600)  # 增加 1 小时（3600 秒）
```

2.2.2.3 日期时间的比较

- `__lt__(other)`：判断当前日期时间是否早于另一个日期时间。

```
if current_datetime < new_datetime:
    print("当前日期时间早于新的日期时间")
```

- `__gt__(other)`：判断当前日期时间是否晚于另一个日期时间。

```
if current_datetime > new_datetime:
    print("当前日期时间晚于新的日期时间")
```

- `__eq__(other)`：判断两个日期时间是否相等。

```
if current_datetime == new_datetime:
    print("两个日期时间相等")
```

2.2.2.4 日期时间的格式化

- `toString(str format)`：将日期时间转换为字符串，使用指定的格式。
  - 常用的格式符有：
    - `yyyy`：四位年份。
    - `MM`：两位月份。
    - `dd`：两位日期。
    - `hh`：两位小时（12 小时制）。
    - `HH`：两位小时（24 小时制）。
    - `mm`：两位分钟。
    - `ss`：两位秒数。

```
formatted_datetime = current_datetime.toString("yyyy-MM-dd HH:mm:ss")
print(f"格式化后的日期时间: {formatted_datetime}")
```

华清远见元宇宙实验中心 yyzlab.com.cn

- `fromString(str string, str format)`：从字符串解析日期时间，使用指定的格式。

```
parsed_datetime = QDateTime.fromString("2023-10-01 12:00:00", "yyyy-MM-dd HH:mm:ss")
print(f"解析后的日期时间: {parsed_datetime.toString()}")
```

2.2.2.5 时区处理

- `toTimeZone(QTimeZone timezone)`：将日期时间转换为指定时区的时间。

```
from PyQt5.QtCore import QTimeZone

utc_time = current_datetime.toTimeZone(QTimeZone.utc())  # 转换为 UTC 时间
print(f"UTC 时间: {utc_time.toString()}")
```

- `offsetFromUtc()`：获取当前日期时间与 UTC 时间的偏移量（以秒为单位）。

```
offset = current_datetime.offsetFromUtc()
print(f"当前时间与 UTC 时间的偏移量: {offset} 秒")
```

2.2.3 典型应用场景

- **获取当前日期和时间**：可以获取系统当前的日期和时间。
- **日期时间的加减**：可以对日期时间进行加减操作，例如增加或减少天数、小时数等。
- **日期时间的比较**：可以比较两个日期时间的先后顺序。
- **日期时间的格式化**：可以将日期时间转换为字符串，或者从字符串解析日期时间。
- **时区处理**：可以处理不同时区的日期时间。

2.3 QLCDNumber

在 PyQt5 中，`QLCDNumber` 是一个用于显示数字的控件，它模仿了传统的液晶显示屏（LCD）样式。`QLCDNumber` 属于 `PyQt5.QtWidgets` 模块，通常用于显示数字或简单的字符，如时间、计数器等。它的外观类似于电子设备上的 LCD 显示屏，具有简洁和清晰的视觉效果。

`QLCDNumber` 的主要作用是以 LCD 样式显示数字或简单的字符。它可以用于以下场景：

- **显示数字**：如计时器、计数器、温度、速度等。
- **显示简单字符**：可以显示一些简单的字符，如十六进制数字、小数点等。
- **动态更新**：可以动态更新显示的内容，适合实时数据展示。

2.3.1 `QLCDNumber` 的导入方式

```
from PyQt5.QtWidgets import QLCDNumber
```

2.3.2 常用方法

2.3.2.1 设置显示的数字

- `display(int | float | str)`：设置要显示的数字或字符串。

```
lcd.display(123)  # 显示数字 123
lcd.display("HELLO")  # 显示字符串 "HELLO"
```

作者更新|完整版实验中心 yyzlab.com.cn

2.3.2.2 设置显示位数

- `setDigitCount(int count)`：设置 LCD 显示的最大位数。

```
lcd.setDigitCount(8)  # 设置最多显示 8 位数字
```

2.3.2.3 设置显示模式

- `setMode(QLCDNumber.Mode mode)`：设置 LCD 的显示模式。
  - `QLCDNumber.Dec`：十进制模式（默认）。
  - `QLCDNumber.Hex`：十六进制模式。
  - `QLCDNumber.Oct`：八进制模式。
  - `QLCDNumber.Bin`：二进制模式。

```
lcd.setMode(QLCDNumber.Hex)  # 设置为十六进制模式
```

2.3.2.4 设置分段样式

- `setSegmentStyle(QLCDNumber.SegmentStyle style)`：设置 LCD 的分段样式。
  - `QLCDNumber.Outline`：轮廓样式（默认）。
  - `QLCDNumber.Filled`：填充样式。
  - `QLCDNumber.Flat`：平面样式。

```
lcd.setSegmentStyle(QLCDNumber.Filled)  # 设置为填充样式
```

2.3.2.5 获取当前显示的值

- `intValue()`：获取当前显示的整数值。

```
value = lcd.intValue()
print(f"当前显示的整数值: {value}")
```

- `value()`：获取当前显示的浮点数值。

```
value = lcd.value()
print(f"当前显示的浮点数值: {value}")
```

2.3.2.6 溢出处理

- `checkOverflow(float value)`：检查是否发生溢出（即显示的数字超出了 LCD 的位数限制）。

```
if lcd.checkOverflow(99999):
    print("发生溢出")
```

2.3.3 典型应用场景

- **显示数字**：如计时器、计数器、温度、速度等。
- **显示简单字符**：可以显示一些简单的字符，如十六进制数字、小数点等。
- **动态更新**：可以动态更新显示的内容，适合实时数据展示。

华清远见元宇宙实验中心 yyzlab.com.cn

2.4 QTime

在 PyQt5 中，`QTime` 是一个用于处理时间的类。它属于 `PyQt5.QtCore` 模块，主要用于表示和操作一天中的时间（小时、分钟、秒和毫秒）。与 `QDateTime` 不同，`QTime` 只关注时间部分，而不涉及日期。

`QTime` 的主要作用是处理时间相关的操作，例如：

- **获取当前时间**：可以获取系统当前的时间。
- **时间的加减**：可以对时间进行加减操作，例如增加或减少秒数、分钟数等。
- **时间的比较**：可以比较两个时间的先后顺序。
- **时间的格式化**：可以将时间转换为字符串，或者从字符串解析时间。
- **时间间隔计算**：可以计算两个时间之间的间隔。

2.4.1 `QTime` 的导入方式

```
from PyQt5.QtCore import QTime
```

2.4.2 常用方法

2.4.2.1 创建 `QTime` 对象

- `QTime(int h, int m, int s=0, int ms=0)`：通过指定的小时、分钟、秒和毫秒创建 `QTime` 对象。

  ```
  time = QTime(12, 30, 45)  # 创建时间为 12:30:45
  ```

- `currentTime()`：获取系统当前的时间。

  ```
  current_time = QTime.currentTime()
  print(f"当前时间: {current_time.toString()}")
  ```

2.4.2.2 时间的加减

- `addSecs(int secs)`：在当前时间的基础上增加或减少秒数。

  ```
  new_time = current_time.addSecs(3600)  # 增加 1 小时（3600 秒）
  ```

- `addMSecs(int ms)`：在当前时间的基础上增加或减少毫秒数。

  ```
  new_time = current_time.addMSecs(5000)  # 增加 5 秒（5000 毫秒）
  ```

2.4.2.3 时间的比较

- `__lt__(other)`：判断当前时间是否早于另一个时间。

  ```
  if current_time < new_time:
      print("当前时间早于新的时间")
  ```

- `__gt__(other)`：判断当前时间是否晚于另一个时间。

华清远见元宇宙实验中心 yyzlab.com.cn

```
if current_time > new_time:
    print("当前时间晚于新的时间")
```

- `__eq__(other)`：判断两个时间是否相等。

```
if current_time == new_time:
    print("两个时间相等")
```

2.4.2.4 时间的格式化

- `toString(str format)`：将时间转换为字符串，使用指定的格式。
  - 常用的格式符有：
    - `hh`：两位小时（12 小时制）。
    - `HH`：两位小时（24 小时制）。
    - `mm`：两位分钟。
    - `ss`：两位秒数。
    - `zzz`：三位毫秒数。

```
formatted_time = current_time.toString("HH:mm:ss")
print(f"格式化后的时间: {formatted_time}")
```

- `fromString(str string, str format)`：从字符串解析时间，使用指定的格式。

```
parsed_time = QTime.fromString("12:30:45", "HH:mm:ss")
print(f"解析后的时间: {parsed_time.toString()}")
```

2.4.2.5 时间间隔计算

- `secsTo(QTime time)`：计算当前时间与另一个时间之间的秒数差。

```
seconds_diff = current_time.secsTo(new_time)
print(f"两个时间之间的秒数差: {seconds_diff} 秒")
```

- `msecsTo(QTime time)`：计算当前时间与另一个时间之间的毫秒数差。

```
milliseconds_diff = current_time.msecsTo(new_time)
print(f"两个时间之间的毫秒数差: {milliseconds_diff} 毫秒")
```

2.4.2.6 检查时间是否有效

- `isValid()`：检查时间是否有效（即是否在合理范围内）。

```
if time.isValid():
    print("时间有效")
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.4.3 典型应用场景

- **获取当前时间**：可以获取系统当前的时间。
- **时间的加减**：可以对时间进行加减操作，例如增加或减少秒数、分钟数等。
- **时间的比较**：可以比较两个时间的先后顺序。
- **时间的格式化**：可以将时间转换为字符串，或者从字符串解析时间。
- **时间间隔计算**：可以计算两个时间之间的间隔。

华清远见|元宇宙实验中心
yyzlab.com.cn